

Architektury systemów komputerowych: Reference

Data: 2026-06-11

Zakres: 1 - 8

1. Logika cyfrowa

Układ	Formuła / sygnatura	Opis
Sumatory		
Półsumator	$s = x \wedge y$ $c = x \& y$	Dodaje dwa bity. <code>s</code> to suma, <code>c</code> to bit przeniesienia (carry).
Pełny sumator (FA)	$s = x \wedge y \wedge ci$ $co = x \& y ci \& (x \wedge y)$	Jak półsumator, ale przyjmuje też przeniesienie wejściowe <code>ci</code> .
Sumator n-bitowy	$n \times FA$	Kaskada (ripple-carry): wyjście C_i trafia na wejście kolejnego FA. Ostatnie <code>co</code> to Carry Out całości.

Układy kombinacyjne		
Dekoder	$n \rightarrow 2^n$	Wejście koduje liczbę k . Na wyjściu zapalony jest tylko bit nr k .
Multiplekser	$2^n + n \rightarrow 1$	2^n bitów danych + n bitów sterujących S . Na wyjście przechodzi S -ty bit danych.
ALU	$A, B, f \rightarrow C$	Dekoder na bitach f wybiera operację (np. $A + B$, $A B$, $A \& B$); multipleksowanie wyników brankami AND/OR.

Układy sekwencyjne (pamięć)		
Przerzutnik S-R	<code>S</code> - set, <code>R</code> - reset	Przechowuje 1 bit (Q). <code>S=1</code> ustawia $Q = 1$, <code>R=1</code> zeruje, <code>S=R=0</code> trzyma stan. Zbudowany z dwóch sprzężonych <code>NOR</code> -ów.
Sterowany poziomem	aktywny gdy <code>CLK</code> = 1	Wejścia <code>S</code> / <code>R</code> bramkowane AND-em z zegarem. Stan zmienia się tylko przy <code>CLK=1</code> .
Sterowany zboczem	zbocze 1 → 0	Dwa przerzutniki poziomowe w kaskadzie (master-slave), drugi z zanegowanym zegarem. Stan przepisuje się w momencie zmiany zegara.

2. Bity, bajty i typy danych

1B = 8b, hex = 4b ⇒ 1B ∈ [0x00; 0xFF]. Np. 15213 = 0x3B6D.

Typ	char	short	int	long	float	double	void*
x86-64	1	2	4	8	4	8	8

3. Kodowanie liczb i konwersja

$$\begin{aligned} \text{B2U}(X) &= \sum_{i=0}^{w-1} x_i 2^i & \text{B2T}(X) &= -x_{w-1} 2^{w-1} + \sum_{i=0}^{w-2} x_i 2^i \\ \text{T2U}(x) &= x < 0 ? x + 2^w : x & \text{U2T}(u) &= u > \text{TMax} ? u - 2^w : u \end{aligned}$$

Skróty: `B` = bity, `U` = unsigned, `T` = ze znakiem (kod U2). Stąd `B2U` = bity → unsigned, podobnie `U2T`, `UMax`, `TMax`, `UAdd`, `TAdd`.

Stała	Bity	Wartość
UMax	11...1	$2^w - 1$
TMax	01...1	$2^{w-1} - 1$ (<code>INT_MAX</code>)
TMin	10...0	-2^{w-1} (<code>INT_MIN</code>)
−1	11...1	te same bity co UMax

Promocja w C: `unsigned` i `int` w jednym wyrażeniu/porównaniu (`< > ==`) → `int` promowany do `unsigned` (bity bez zmian, $-1 \rightarrow \text{UMax}$).

4. Rozszerzanie, obcinanie i przesunięcia

Rozszerz. 0 (<code>zext</code>)	<code>unsigned</code> : dopisuje 0 z góry
Rozszerz. znak (<code>sext</code>)	<code>signed</code> : powiela bit znaku (MSB)
Obcięcie (<code>trunc</code>)	$u \bmod 2^w$; może zmienić znak
<code>x << k</code>	$x \cdot 2^k$ (sgn./uns.)
<code>u >> k</code>	$\lfloor u/2^k \rfloor$ – logiczne (zera)
<code>x >> k</code>	arytmetyczne (kopia MSB), zaokr. do $-\infty$
$x/2^k$ do 0	<code>(x + (1<<k)-1) >> k</code>

Strength red.: `x*24` → `(x<<5)-(x<<3)`.

5. Operacje, overflow i endianness

UAdd	$(u + v) \bmod 2^w$; carry ignorowane
TAdd	bitowo = UAdd; różne znaki nie przepelniają
Nadmiar (+)	$u, v > 0$, a wynik < 0 (wynik -2^w)
Nadmiar (-)	$u, v < 0$, a wynik ≥ 0 (wynik $+2^w$)
Negacja U2	<code>-x = ~x+1</code> ; <code>~TMin=TMin</code> , <code>-0=0</code>
Wykr. nadmiaru (<code>s=x+y</code>)	<code>((s^x)&(s^y))>>(w-1)</code>
Maska / abs	<code>m=x>>31; abs=(x^m)-m</code>

Pamięć: adres wskazuje najmłodszy bajt słowa. Napis = `char[]` ASCII + `0x00` (niezależny od endianness).

Schemat	Najniższy adres	0x0123
Big Endian	MSB	[01][23]
Little Endian	LSB	[23][01]

6. Zmiennoprzecinkowe (IEEE 754)

$V = (-1)^s \times M \times 2^E$ Bias = $2^{k-1} - 1$				
Format	bity	s	exp (<i>k</i>)	frac (<i>n</i>)
Single (float)	32	1	8 (Bias=127)	23
Double (double)	64	1	11 (Bias=1023)	52

Kategorie wartości

Kategoria	exp	Własności
Znormalizowane	!= 0...0 != 1...1	$E = \text{exp} - \text{Bias}$. Ukryta jedynka: $M = 1.\text{frac}$. Najczęściej ułożone wokół 0.
Zdenormaliz.	== 0...0	$E = 1 - \text{Bias}$. Brak jedynki: $M = 0.\text{frac}$. Reprezentują +0.0 i −0.0 (frac = 0) oraz liczby bardzo bliskie zera (stopniowy underflow).
Specjalne	== 1...1	Gdy frac == 0, to $+\infty$ lub $-\infty$ (nadmiar/dzielenie przez 0). Gdy frac $\neq$ 0, to NaN (np. $\sqrt{-1}$, $\infty - \infty$).

Zaokrąglenie GRS i Round-to-Even

Domyślny tryb to **Round-to-Even** (zapobiega błędowi statycznemu przy sumowaniu).

... x x G R S S S ... G – ostatni zachowany, R – pierwszy usunięty, S – OR reszty		
Warunek	Ułamek	Akcja
R=0	< 0.5	W dół (odcięcie)
R=1, S=1	> 0.5	W górę (+1 do G)
R=1, S=0	= 0.5	Do parzystej: w górę gdy G=1 , w dół gdy G=0

Własności matematyczne i rzutowanie

- Brak łączności:** $(a + b) + c \neq a + (b + c)$, przez zaokrąglenia i utratę danych.
- Brak rozdzielności:** $a(b + c) \neq ab + ac$.
- Rzutowanie**

int

 $\rightarrow$

float

 : Może stracić precyzję (zaokrągła zgodnie z trybem).
- Rzutowanie**

int

 $\rightarrow$

double

 : Dokładne i bezstratne (bo 52 bity mantysy > 32 bity inta).
- Rzutowanie**

float/double

 $\rightarrow$

int

 : Obcina ułamek w stronę 0. Jeśli poza zakresem lub NaN $\rightarrow$ z reguły zwraca

TMin

 (0x80000000).

7. Instrukcja leaq (Load Effective Address)

Instrukcja **obliczeniowa**, nie pamięciowa. Ładuje **obliczony adres**, a nie wartość z pamięci: `leaq Src, Dst` $\rightarrow$ `Dst = addr(Src)` .

Zastosowania:

- Pobieranie adresu zmiennej:** C-owe `p = &x[i]` .
- Szybka arytmetyka bez flag:** Obliczenia $x + k \cdot y$ dla stałych $k \in \{1, 2, 4, 8\}$.

Przykład: Optymalizacja `x * 12` :

```
leaq (%rdi, %rdi, 2), %rax ; t = x + 2*x = 3*x
salq $2, %rax ; return t << 2 (czyli 3*x * 4 = 12*x)
```

8. Rejestry x86_64

%rax	%eax	%ax %ah %al	%rbx	%ebx	%bx %bh %bl
%rcx	%ecx	%cx %ch %cl	%rdx	%edx	%dx %dh %dl

%rsi	%esi	%si %sil	%rdi	%edi	%di %dil
%rbp	%ebp	%bp %bpl	%rsp	%esp	%sp %spl
%r8	%r8d	%r8w %r8b	%r9	%r9d	%r9w %r9b

Uwaga: Rejestry od `%r10` do `%r15` używają schematu:

%r10

 ,

%r10d

 ,

%r10w

 ,

%r10b

 .

Podział ról rejestrów

%rax	Akumulator. Główny rejestr arytmetyczny. Przechowuje wartość zwracaną .
%rdi, %rsi, %rdx, %rcx, %r8, %r9	Kolejno: od 1. do 6. argumentu funkcji . Caller-saved.
%rsp	Wskaźnik stosu (SP). Wskazuje wierzchołek ramki.
%rbp	Wskaźnik bazy (BP). Początek ramki stosu. Callee-saved.
%rbx, %r12- %r15	Ogólnego przeznaczenia. Wszystkie callee-saved .
%rip	Wskaźnik instrukcji (Program Counter).

9. Tryby adresowania (operands)

Tryb	Format	Obliczanie adresu
Natychnmiastowy (Imm)	\$Imm	Imm
Rejestrowy (Reg)	%Ra	%Ra
Bezpośredni (Mem)	Imm	M[Imm]
Pośredni (Mem)	(%Rb)	M[%Rb]
Z przesunięciem	D(%Rb)	M[%Rb + D]
Skalowany (Ind/scaled)	D(%Rb, %Ri, S)	M[%Rb + %Ri * S + D]
Skalowany (baseless)	(, %Ri, S)	M[%Ri * S]

Legenda:

Imm

 /

D

 to stała liczbowa,

%Ra

 /

%Rb

 to rejestr bazowy,

%Ri

 to rejestr indeksowy,

a

S

 to skala (tylko: 1, 2, 4 lub 8).

10. Podstawowe instrukcje i arytmetyka (AT&T)

Opcode	Efekt	Opis
<code>mov S, D</code>	$D \leftarrow S$	Kopiuje wartość z S do D.
<code>lea S, D</code>	$D \leftarrow \text{addr}(S)$	Oblicza adres S (bez czytania pamięci).
<code>add S, D</code>	$D \leftarrow D + S$	Dodawanie (ustawia flagi).
<code>sub S, D</code>	$D \leftarrow D - S$	Odejmowanie (ustawia flagi).
<code>imul S, D</code>	$D \leftarrow D * S$	Mnożenie liczb ze znakiem.
<code>sal / shl k, D</code>	$D \ll= k$	Przesunięcie w lewo (mnożenie przez 2^k).
<code>sar k, D</code>	$D \gg= k$	Arytmetyczne w prawo (dzielenie). Kopiuje znak .
<code>shr k, D</code>	$D \gg= k$	Logiczne w prawo. Dopełnia zerami.
<code>and / or S, D</code>	$D \leftarrow D \& / S$	Operacje bitowe (ustawiają flagi).
<code>xor S, D</code>	$D \leftarrow D \wedge S$	Często używane jako <code>xor %rax, %rax</code> do zerowania.

Uwaga o sufiksach: Instrukcje przyjmują sufiks określający rozmiar danych:

`b` (8-bit), `w` (16-bit), `l` (32-bit), `q` (64-bit).

Np. `movl` kopiuje 32 bity (i automatycznie zeruje górną połowę 64-bitowego rejestru!).

11. Rejestry x86_64

<code>%rax</code>	<code>%eax</code>	<code>%ax</code> <code>%ah</code> <code>%al</code>	<code>%rbx</code>	<code>%ebx</code>	<code>%bx</code> <code>%bh</code> <code>%bl</code>
<code>%rcx</code>	<code>%ecx</code>	<code>%cx</code> <code>%ch</code> <code>%cl</code>	<code>%rdx</code>	<code>%edx</code>	<code>%dx</code> <code>%dh</code> <code>%dl</code>
<code>%rsi</code>	<code>%esi</code>	<code>%si</code> <code>%sil</code>	<code>%rdi</code>	<code>%edi</code>	<code>%di</code> <code>%dil</code>
<code>%rbp</code>	<code>%ebp</code>	<code>%bp</code> <code>%bpl</code>	<code>%rsp</code>	<code>%esp</code>	<code>%sp</code> <code>%spl</code>
<code>%r8</code>	<code>%r8d</code>	<code>%r8w</code> <code>%r8b</code>	<code>%r9</code>	<code>%r9d</code>	<code>%r9w</code> <code>%r9b</code>

Uwaga: Rejestry od `%r10` do `%r15` używają schematu:

`%r10`, `%r10d`, `%r10w`, `%r10b`.

Rejestry ogólnego przeznaczenia

<code>%rax</code>	Akumulator. Główny rejestr arytmetyczny. Przechowuje wartość zwracaną z funkcji.
<code>%rbx</code>	Rejestr bazowy. Callee-saved.
<code>%rcx</code>	Licznik w pętlach i przesunięciach bitowych. 4. argument .
<code>%rdx</code>	Rejestr danych. Używany w mnożeniu/dzieleniu. 3. argument .

Indeksy i wskaźniki

<code>%rdi</code>	Indeks docelowy (DI). 1. argument .
<code>%rsi</code>	Indeks źródłowy (SI). 2. argument .

`%rsp` Wskaźnik stosu (SP). Wskazuje wierzchołek ramki. Modyfikowany przez `push` / `pop` / `call` / `ret`.

`%rbp` Wskaźnik bazy (BP). Początek ramki stosu. Callee-saved.

Nowe i specjalne rejestry

`%r8-9` Kolejno: **5. i 6. argument**. Caller-saved.

`%r10` oraz `%r11` Rejestry tymczasowe. Caller-saved.

`%r12` do `%r15` Ogólnego przeznaczenia. Wszystkie **callee-saved**.

`%rip` Wskaźnik instrukcji (IP). Następną instrukcja. Modyfikowany przez skoki i wywołania.

`%rflags` Rejestr flag statusu. Zmieniany przez `cmp`, `test`.

Rejestry wektorowe (zmiennoprzecinkowe)

`%xmm0` do `%xmm15` 128-bitowe rejestry. `%xmm0` to **wartość zwracana** (float/double). `%xmm0-%xmm7` to pierwsze **8 argumentów**. Wszystkie są **caller-saved**.

`%ymm0` do `%ymm15` 256-bitowe rozszerzenie rejestrów XMM. Dzielią z nimi najmłodsze 128 bitów.

12. Flagi stanu (Rejestr `%rflags`)

ZF

Zero. Wynik to 0 (np. argumenty są równe).

SF

Sign. Wynik jest ujemny (MSB = 1).

CF

Carry. Przepelnienie dla liczb **bez znaku** (unsigned).

OF

Overflow. Przepelnienie dla liczb **ze znakiem** (signed).

13. Tryby adresowania (operandy)

Tryb	Format	Obliczanie adresu
Natychmiastowy (Immediate)	<code>\$Imm</code>	<code>Imm</code>
Rejestrowy (Register)	<code>%Ra</code>	<code>%Ra</code>
Bezpośredni (Direct)	<code>Imm</code>	<code>M[Imm]</code>
Pośredni (Indirect)	<code>(%Rb)</code>	<code>M[%Rb]</code>
Z przesunięciem (Indirect displacement)	<code>D(%Rb)</code>	<code>M[%Rb + D]</code>
Skalowany (Indirect scaled-index)	<code>D(%Rb, %Ri, S)</code>	<code>M[%Rb+%Ri*S+D]</code>
Skalowany (baseless)	<code>(, %Ri, S)</code>	<code>M[%Ri * S]</code>

Legenda: `Imm` / `D` to stała liczbowa, `%Ra` / `%Rb` to rejestr bazowy, `%Ri` to rejestr indeksowy, `a` / `S` to skala (tylko wartości: 1, 2, 4 lub 8).

14. Konwencja wywoływań (System V AMD64 ABI)

Przekazywanie argumentów

`%rdi`

`→`

`%rsi`

`→`

`%rdx`

`→`

`%rcx`

`→`

`%r8`

`→`

`%r9`

Argumenty 7+ Na stosie od końca.

Wynik Zawsze w `%rax`.

Ochrona rejestrów (ABI)

Caller-saved
(Można niszczyć)

`%rax` , `%rdi` , `%rsi` , `%rdx` , `%rcx` , `%r8` - `%r11`

Callee-saved
(Musi przywrócić)

`%rbx` , `%rbp` , `%r12` - `%r15`

Ramka Stosu

Start (alokacja)

```
push %rbp
mov %rsp, %rbp
sub $32, %rsp
```

Koniec (sprzątanie)

```
leave
ret
/* równowaznie:
* mov %rbp, %rsp
* pop %rbp
* ret */
```

15. Struktury i wyrównanie

1. **Wyrównanie pola (K):** Zmienna K -bajtowa pod adresem p mod $K = 0$.

2. **Rozmiar całkowity:** Całkowity `sizeof` struktury musi być podzielny przez największe wyrównanie w strukturze ($K_{\max}$).

```
struct {
    char c; // 1 bajt (offset 0)
           // [3 bajty wewn. paddingu]
    int i; // 4 bajty (offset 4)
    short s; // 2 bajty (offset 8)
           // [2 bajty zewn. paddingu]
} // K_max = 4. Rozmiar: 1+3+4+2+2 = 12 bajtów.
```

16. Najważniejsze instrukcje (AT&T)		
Opcode	Efekt	Opis
Przesyłanie danych		
<code>mov S, D</code>	$D \leftarrow S$	Kopiuje wartość z S do D.
<code>push S</code>	$\%rsp - = 8$ $M[\%rsp] \leftarrow S$	Odkłada na stos.
<code>pop D</code>	$D \leftarrow M[\%rsp]$ $\%rsp + = 8$	Zdejmuje ze stosu do D.
<code>leave</code>	$\%rsp \leftarrow \%rbp$ <code>pop %rbp</code>	Sprząta ramkę stosu.
Adresowanie i arytmetyka		
<code>lea S, D</code>	$D \leftarrow \text{addr}(S)$	Oblicza adres S (bez czytania pamięci).
<code>add S, D</code>	$D \leftarrow D + S$	Dodawanie (ustawia flagi).
<code>sub S, D</code>	$D \leftarrow D - S$	Odejmowanie (ustawia flagi).
<code>imul S, D</code>	$D \leftarrow D * S$	Mnożenie liczb ze znakiem.
<code>inc / dec D</code>	$D \leftarrow D \pm 1$	Inkrementacja / dekrementacja.
<code>neg D</code>	$D \leftarrow -D$	Negacja arytmetyczna (jak w U2).
Dzielenie (zawsze w parze)		
<code>cqto</code>	$\%rdx:\%rax \leftarrow \text{sgn_ext}(\%rax)$	Uwaga: Rozszerza <code>%rax</code> do 128-bit przed <code>idivq</code> .
<code>idiv S</code>	$\%rax \leftarrow \text{wynik}$ $\%rdx \leftarrow \text{reszta}$	Dzielenie <code>%rdx:%rax</code> przez S. Wynik w <code>%rax</code> , reszta w <code>%rdx</code> .
Logiczne i bitowe		
<code>and / or S, D</code>	$D \leftarrow D \& / S$	Operacje bitowe (ustawiają flagi).
<code>xor S, D</code>	$D \leftarrow D \wedge S$	Często używane jako <code>xor %rax, %rax</code> do zerowania.
<code>not D</code>	$D \leftarrow \sim D$	Negacja bitowa (odwrócenie bitów).
<code>sal / shl k, D</code>	$D \ll = k$	Przesunięcie w lewo (mnożenie przez 2^k).
<code>sar k, D</code>	$D \gg = k$	Arytmetyczne w prawo (dzielenie). Kopiuje znak.

Patryk Pujanek | gh: @nihiL7331

Opcode	Efekt	Opis
<code>shr k, D</code>	$D \gg= k$	Logiczne w prawo. Dopełnia zerami.
Porównania i skoki		
<code>cmp S1, S2</code>	$S2 - S1$	Ustawia flagi jak odejmowanie, nie zapisuje wyniku.
<code>test S1, S2</code>	$S2 \& S1$	Ustawia flagi jak AND (np. test czy rejestr jest zerem).
<code>jX dest</code>	$\text{if}(X) \%rip \leftarrow \text{dest}$	Skok warunkowy (X = warunek).
<code>setX D</code>	$\text{if}(X) D \leftarrow 1$	Ustawia bajt (np. <code>%al</code>) na 0 lub 1 na podstawie flag.
<code>cmovX S, D</code>	$\text{if}(X) D \leftarrow S$	Warunkowe kopiowanie (optymalizacja zamiast skoku).
<code>call / ret</code>		Skok do funkcji / Powrót (obsługa stosu i <code>%rip</code>).

Operacje wektorowe		
<code>movaps / movups S, D</code>	$D \leftarrow S$	Kopiuje 128-bitów. <code>aps</code> (aligned) – wymaga wyrównania w pamięci, <code>ups</code> nie.
<code>movsd / movss S, D</code>	$D \leftarrow S$	Kopiuje pojedynczą wartość skalarną (<code>double</code> / (s) <code>float</code>).
<code>add/sub/mul/div pd/ps</code>	$D \leftarrow D \text{ op } S$	Arytmetyka wektorowa. Wykonuje operację na wielu parach równocześnie.
<code>add/sub/mul/div sd/ss</code>	$D \leftarrow D \text{ op } S$	Arytmetyka skalarna. Modyfikuje tylko najmłodszą wartość w rejestrze.
<code>vxorps S1, S2, D</code>	$D \leftarrow S2 \wedge S1$	Wektorowy XOR .
<code>v... (np. vmulss)</code>	$D \leftarrow S2 \text{ op } S1$	Przedrostek <code>v</code> wprowadza 3 argumenty (źródło 1, źródło 2, cel). Nie niszczy danych wejściowych.
<code>vfmadd231ss S1, S2, D</code>	$D \leftarrow S2 * S1 + D$	Fused Multiply-Add . Mnoży i dodaje. Cyfry <code>231</code> określają, że mnożymy dwa źródła, a wynik dodajemy do <code>D</code> .

Uwaga o sufiksach: Instrukcje mogą przyjmować sufiks określający rozmiar danych: `b` (byte, 8-bit), `w` (word, 16-bit), `l` (long, 32-bit), `q` (quadword, 64-bit). Np. `movq` kopiuje 64 bity, a `movl` kopiuje 32 bity (i zeruje górną połowę rejestru!).

17. Sufiksy warunkowe (dla <code>jX</code> , <code>setX</code> , <code>cmovX</code>)		
Sufiks	Synonim	Znaczenie / Warunek
Równość i znak (<code>ZF</code> , <code>SF</code>)		
<code>e</code>	<code>z</code>	Equal / Zero (równe / wynik to 0)
<code>ne</code>	<code>nz</code>	Not Equal / Not Zero (nierówne)
<code>s</code>		Sign (wynik ujemny, <code>SF=1</code>)
<code>ns</code>		Not Sign (wynik dodatni lub 0, <code>SF=0</code>)

Liczby ze znakiem (signed)	
<code>g</code>	Greater
<code>ge</code>	Greater or Equal
<code>l</code>	Less
<code>le</code>	Less or Equal

Liczby bez znaku (unsigned)	
<code>a</code>	Above (powyżej: <code>></code>)
<code>ae</code>	<code>nc</code> Above or Equal / No Carry (większe równe: <code>>=</code>)
<code>b</code>	<code>c</code> Below / Carry (poniżej: <code><</code>)
<code>be</code>	Below or Equal (mniejsze równe: <code><=</code>)

18. Optymalizacje i atrybuty	
Podstawowe techniki	
Strength red.	Zamiana drogich operacji na tańsze (np. <code>mul</code> $\rightarrow$ <code>lea</code> / <code>shl</code>).
Loop unroll.	Rozwinięcie pętli (np. skok co 4. iterację). Mniej skoków = mniejszy narzut.
Code motion	Wyrzucenie obliczeń niezmienników (np. stałych w pętli) przed pętlę.
Common subexpr. elim.	Obliczenie wspólnego fragmentu tylko raz.
Const folding	Obliczanie stałych wyrażeń (np. <code>2+2</code>) w czasie kompilacji.
Branch predict	Sprzętowe przewidywanie skoków. By unikać kar, można użyć <code>cmovX</code> .
Inlining	Wklejenie ciała funkcji w miejsce wywołania. Eliminuje narzut <code>call</code> / <code>ret</code> .

Analiza pamięci i atrybuty GCC	
Aliasing	Problem nakładania się wskaźników. Keyword <code>restrict</code> , pomaga w optymalizacji.
pure / const	Funkcja bez skutków ubocznych (<code>const</code>) dodatkowo nie czyta zmiennych globalnych).

19. Bezpieczeństwo kodu i exploity	
Zagrożenia i błędy	
Buffer overflow	Zapis poza limit bufora. Nadpisuje sąsiadującą pamięć.
Seg fault	Próba dostępu bez uprawnień (np. odczyt <code>NULL</code> , modyfikacja read-only).

Stack smashing	Atak nadpisujący adres powrotu na stosie, by przejąć kontrolę po <code>ret</code> .
ROP (Gadżety)	Łączenie legalnych instrukcji kończących się <code>ret</code> w celu ominięcia zabezpieczeń.
Mechanizmy obronne	
Stack canaries	Losowa wartość ułożona przed adresem powrotu. Weryfikowana przed <code>ret</code> .
Nonexec code segments	Stos dostaje sprzętowy zakaz wykonywania kodu (tylko R/W).
Rand. stack offsets	Losowanie adresów stosu przy każdym starcie.